

Universidade Federal
de São João del-Rei

Universidade Federal de São João Del Rei
Curso de Graduação em Ciência da Computação

Caio Augusto Silva dos Reis

Relatório de Iniciação Científica: Vitrine um Sistema de Publicação de Conteúdo Científico

São João Del Rei, Minas Gerais

2026

Caio Augusto Silva dos Reis

Relatório de Iniciação Científica: Vitrine um Sistema de Publicação de Conteúdo Científico

Relatório Final de Iniciação Científica apresentado ao Departamento de Ciência da Computação da Universidade Federal de São João Del Rei, referente às atividades de pesquisa desenvolvidas de setembro de 2025 a fevereiro de 2026. O trabalho descreve a concepção e implementação do sistema Vitrine, sob orientação do Prof. Dr. Matheus Carvalho Viana.

Orientador: Prof. Dr. Matheus Carvalho Viana

São João Del Rei, Minas Gerais

2026

Resumo

Este relatório apresenta o desenvolvimento do Vitrine, uma plataforma de gerenciamento de conteúdo (CMS) open source projetada para democratizar a publicação independente. O problema central reside na barreira tecnológica enfrentada por docentes e pesquisadores que, ao buscarem autonomia digital, esbarram na complexidade de infraestruturas tradicionais ou na rigidez de plataformas proprietárias. A pesquisa, de natureza aplicada e caráter exploratório-descritivo, utilizou uma metodologia de desenvolvimento experimental e incremental para a construção de um artefato tecnológico moderno. O sistema foi construido sobre o framework Next.js, integrando o ORM Prisma com banco de dados SQLite para garantir a portabilidade dos dados e simplicidade no deployment. Os resultados alcançados incluem a implementação de um editor Markdown com suporte a fórmulas em LaTeX e um sistema de personalização visual dinâmica via variáveis CSS, refletidas instantaneamente através de Incremental Static Regeneration (ISR). A solução equilibra o alto desempenho de sites estáticos com a flexibilidade de sistemas dinâmicos, oferecendo uma ferramenta extensível que elimina o trabalho repetitivo de configuração para produtores de conteúdo acadêmico.

Palavras-chave: CMS Acadêmico; Next.js; Open Source; Markdown; Iniciação Científica.

Sumário

1 INTRODUÇÃO 5
1.1 Contextualização 5
1.2 Motivação 5
1.3 Objetivos 5
1.3.1 Objetivo Geral 5
1.3.2 Objetivos Específicos 6
1.4 Justificativa 6
1.5 Organização do Relatório Final 6

2 FUNDAMENTAÇÃO TEÓRICA E TRABALHOS RELACIONADOS 8
2.1 Referencial Tecnológico 8
2.1.1 Next.js e o Paradigma Jamstack 8
2.1.2 Persistência de Dados com SQLite e Prisma 8
2.1.3 Segurança e Autenticação 8
2.2 Análise de Trabalhos Relacionados 8
2.2.1 Limitações de Plataformas Existentes 9
2.2.2 O Diferencial do Vitrine 9

3 METODOLOGIA 10
3.1 Classificação da Pesquisa 10
3.2 Procedimentos Técnicos 10
3.3 Etapas do Desenvolvimento 10
3.4 Cronograma Realizado (Setembro/2025 – Fevereiro/2026) 11
3.5 Requisitos do Sistema 12
3.6 Modelagem do Sistema 12
3.6.1 Diagrama de Casos de Uso 12
3.6.2 Modelagem de Dados (ER) 12
3.7 Arquitetura Técnica 15
3.8 Interfaces e Prototipagem 15
3.8.1 Visualização do Sistema com Conteúdo (Instância FromCaio) 15
3.8.2 Gestão e Organização de Conteúdos 16
3.8.3 Customização Dinâmica 17
3.8.4 Editor de Escrita Científica 17
3.9 Implementação das Funcionalidades Centrais 18
3.9.1 Sistema de Cores e ISR 18

4 CONCLUSÃO 19
4.1 Resultados Alcançados 19
4.2 Limitações e Desafios 19
4.3 Considerações Finais e Trabalhos Futuros 20

REFERÊNCIAS 21

1 Introdução

1.1 Contextualização

A disseminação de conhecimento acadêmico encontrou na web seu principal canal de expansão (1). No entanto, para docentes e pesquisadores — especialmente de áreas não tecnológicas —, a criação de um espaço próprio para publicação de artigos e materiais didáticos ainda esbarra em um dilema: a dependência de plataformas proprietárias e rígidas ou a complexidade de gerenciar infraestruturas de sistemas de gerenciamento de conteúdo (*Content Management Systems* - CMS) tradicionais (2).

Muitos educadores recorrem a redes sociais ou plataformas de terceiros para compartilhar seus estudos, o que acarreta na perda de controle sobre a identidade visual, na dificuldade de formatação de elementos técnicos — como fórmulas matemáticas e blocos de código — e na fragilidade quanto à perenidade do conteúdo. Nesse cenário, surge o interesse em desenvolver uma ferramenta que una a autonomia de um domínio próprio com a simplicidade de um serviço pronto para uso, reduzindo as barreiras tecnológicas.

1.2 Motivação

O projeto **Vitrine** nasce da convicção de que cada educador deve ter a capacidade de possuir sua própria "vitrine"de conhecimento, de forma independente e personalizada. A motivação principal reside em transformar o processo de implantação (*deployment*) em um ato acessível, permitindo que a barreira técnica não seja um impedimento para a presença digital qualificada do docente.

Ao oferecer uma solução *open source*, modular e versátil, o projeto busca devolver ao autor o controle total sobre sua produção intelectual e a estética de sua apresentação. Conforme destacado por 3, sistemas de informação eficazes devem servir como facilitadores desse processo, e não como obstáculos.

1.3 Objetivos

1.3.1 Objetivo Geral

Desenvolver o **Vitrine**, uma plataforma CMS *open source*, modular e de fácil implantação, baseada no framework Next.js, voltada para a criação de blogs e portfólios acadêmicos por professores e pesquisadores.

1.3.2 Objetivos Específicos

- Viabilizar a publicação independente por meio de um processo de *clonagem* e *deployment* simplificado;
- Implementar um editor de texto baseado em *markdown* que suporte necessidades da escrita científica, como realce de sintaxe e fórmulas em *LaTeX*;
- Prover um sistema de personalização visual dinâmica que permita a alteração da paleta de cores via interface administrativa, sem necessidade de novos ciclos de compilação (*build*);
- Adotar uma arquitetura de dados portátil utilizando o ORM Prisma e o banco de dados SQLite (4);
- Integrar um sistema de autenticação simplificado através do Auth.js para garantir a segurança no gerenciamento de conteúdos.

1.4 Justificativa

A proposta deste trabalho justifica-se pela busca do equilíbrio entre modernidade tecnológica e portabilidade. O uso do **Next.js** permite uma renderização híbrida de alto desempenho, essencial para a indexação em motores de busca (SEO) (5). A escolha do **SQLite** é estratégica por ser um sistema baseado em arquivo, o que simplifica a infraestrutura necessária e reduz custos para o usuário final (4).

Embora o **Vitrine** seja intuitivo para usuários leigos, ele também atende a desenvolvedores ao oferecer uma base sólida que elimina o trabalho repetitivo de configurar estruturas do zero. Customizações podem ser feitas através de variáveis CSS em tempo de execução com persistência em banco de dados e reflexão instantânea via *Incremental Static Regeneration* (ISR) (5).

1.5 Organização do Relatório Final

Este documento detalha o ciclo completo da pesquisa realizada entre setembro de 2025 e fevereiro de 2026, estando organizado da seguinte forma:

- Capítulo 2: Fundamentação teórica sobre React, Next.js e o paradigma Jamstack;
- Capítulo 3: Metodologia de pesquisa aplicada, levantamento de requisitos e modelagem via protótipos de baixa fidelidade;

- Capítulo 3.6.2: Descrição técnica do desenvolvimento do Vitrine e decisões de arquitetura;
- Capítulo 4: Conclusões sobre os resultados obtidos, limitações e propostas de trabalhos futuros.

2 Fundamentação Teórica e Trabalhos Relacionados

2.1 Referencial Tecnológico

O desenvolvimento do sistema Vitrine fundamenta-se em tecnologias modernas de desenvolvimento web que priorizam o desempenho e a simplicidade de manutenção.

2.1.1 Next.js e o Paradigma Jamstack

A plataforma utiliza o framework **Next.js**, que permite uma renderização híbrida de alto desempenho (5). Esta escolha é essencial para garantir a indexação em motores de busca (SEO) e uma experiência de usuário fluida. O projeto alinha-se ao paradigma *Jamstack*, focando na dissociação entre a camada de apresentação e os dados, o que favorece a segurança e a escalabilidade (1).

2.1.2 Persistência de Dados com SQLite e Prisma

Para garantir a portabilidade, o Vitrine utiliza o **SQLite** como sistema de gerenciamento de banco de dados (4). Por ser um sistema baseado em arquivo, o SQLite simplifica drasticamente a infraestrutura necessária, eliminando a necessidade de servidores de banco de dados complexos e reduzindo custos para o usuário final. A interação com os dados é mediada pelo **Prisma ORM**, que provê uma camada de abstração segura e eficiente para a manipulação das tabelas (4).

2.1.3 Segurança e Autenticação

A segurança no gerenciamento de conteúdos é garantida pela integração com o **Auth.js**. Esta ferramenta permite um sistema de autenticação simplificado, protegendo o painel administrativo contra acessos não autorizados.

2.2 Análise de Trabalhos Relacionados

A disseminação de conhecimento acadêmico na web esbarra frequentemente na dependência de plataformas proprietárias ou na complexidade de sistemas de gerenciamento de conteúdo (*Content Management Systems* - CMS) tradicionais (2).

2.2.1 Limitações de Plataformas Existentes

Muitos pesquisadores recorrem a redes sociais ou plataformas de terceiros para compartilhar sua produção (1). No entanto, essas soluções apresentam fragilidades críticas:

- **Perda de Controle**: Dificuldade em manter a integridade da identidade visual e a formatação de elementos técnicos em virtude de regras de plataformas de terceiros, que impõem limites de caracteres e submetem a produção acadêmica à competição visual com conteúdos de naturezas diversas;
- **Barreiras Técnicas**: CMSs tradicionais exigem conhecimentos avançados de infraestrutura para instalação e manutenção (2);
- **Formatação Técnica**: Falta de suporte nativo para elementos essenciais na escrita científica, como fórmulas em *LaTeX* e blocos de código técnico.

2.2.2 O Diferencial do Vitrine

Diferente de soluções rígidas, o Vitrine busca o equilíbrio entre autonomia e simplicidade, posicionando-se como um ponto de destino especializado para a produção docente. A plataforma permite gerenciar a estética visual (variáveis CSS) em tempo de execução com persistência em banco de dados, refletindo alterações instantaneamente via *Incremental Static Regeneration* (ISR) (5).

Esta abordagem garante a integridade de elementos complexos, como fórmulas em *LaTeX* e blocos de código técnico, assegurando que o material mantenha sua precisão pedagógica. Ao contrário de ecossistemas fechados que dificultam a exportação de dados, a arquitetura do Vitrine foca na soberania do autor: a posse integral das informações permite que o pesquisador utilize a ferramenta como sua base central de conhecimento (4).

3 Metodologia

Este capítulo detalha a metodologia de pesquisa e os processos de engenharia de software aplicados no desenvolvimento do sistema Vitrine durante o período de iniciação científica. A abordagem adotada busca unir o rigor acadêmico à aplicação prática de tecnologias modernas para solucionar o problema da publicação acadêmica independente.

3.1 Classificação da Pesquisa

Quanto à sua **natureza**, esta pesquisa é classificada como **aplicada**, visto que objetiva a criação de um artefato tecnológico — um sistema de gerenciamento de conteúdo (CMS) — destinado a resolver o problema prático da presença digital docente. Quanto aos **objetivos**, o trabalho possui caráter **descritivo** e **exploratório**. É exploratório na fase de investigação de tecnologias como o framework Next.js e o paradigma Jamstack, e descritivo ao detalhar as etapas de modelagem, arquitetura e requisitos da solução.

3.2 Procedimentos Técnicos

Para a condução do estudo, foram adotados os seguintes procedimentos:

- **Revisão Bibliográfica**: O foco recaiu sobre arquiteturas web, bancos de dados portáteis (SQLite) e padrões de design para interfaces de usuário, fornecendo o embasamento necessário para as decisões arquiteturais do projeto (1).
- **Desenvolvimento Experimental e Incremental**: O método baseia-se no desenvolvimento de um protótipo funcional através de um processo **iterativo e incremental** (6). Esta abordagem permitiu que funcionalidades complexas fossem validadas de ponta a ponta a cada incremento realizado.

3.3 Etapas do Desenvolvimento

O processo de construção do Vitrine foi estruturado em quatro macroetapas, fundamentadas nos processos clássicos de engenharia de software (6):

1. **Levantamento de Requisitos**: Identificação das necessidades dos usuários (professores e desenvolvedores), resultando na definição de Requisitos Funcionais (RF) e Não-Funcionais (RNF) (6).

2. **Modelagem de Sistemas**: Utilização da linguagem UML e DBML para visualização da lógica do sistema, incluindo Diagramas de Casos de Uso e Modelagem de Dados (ER) (6).

3. **Prototipagem**: Criação de protótipos de **baixa fidelidade** para validar a usabilidade do editor de Markdown e do painel administrativo antes da implementação final (6).

4. **Desenvolvimento Incremental (Sprints)**: Implementação do código-fonte utilizando Next.js e Prisma ORM, abrangendo desde a persistência no SQLite até a renderização no front-end e configuração de cache (ISR) (5, 4).

3.4 Cronograma Realizado (Setembro/2025 – Fevereiro/2026)

O cronograma abaixo reflete o ciclo de execução do projeto, destacando a natureza incremental da implementação:

Tabela 1 – Cronograma de Execução do Projeto Vitrine

Etapa	**Atividade Específica**	**Início**	**Término**
Concepção	Levantamento de Requisitos e Escopo	Set/25	Out/10
	Revisão Bibliográfica e Tecnológica	Set/25	Out/10
Modelagem	Diagramas de Casos de Uso e ER	Out/10	Out/25
	Protótipos de Baixa Fidelidade (UI/UX)	Out/25	Nov/5
	Definição da Arquitetura e Stack	Out/25	Nov/5
Desenvolvimento	Setup do Ambiente e Auth.js	Nov/5	Nov/10
	Módulo de Conteúdo e Markdown	Nov/10	Fev/10
	Módulo de Personalização e ISR	Nov/10	Fev/10
Finalização	Escrita do Relatório Final e Testes	Fev/10	Fev/26
	Entrega do Relatório	Fev/10	Fev/26

3.5 Requisitos do Sistema

A especificação dos requisitos foi a base para o desenvolvimento incremental da plataforma (6):

- **RF01 - Autenticação**: Acesso restrito via Auth.js para segurança no gerenciamento.
- **RF02 - Estrutura Hierárquica**: Suporte a documentos e coleções aninhadas.
- **RF03 - Ordenação**: Atributo de ordem relativa entre elementos irmãos.
- **RF04 - Personalização**: Interface para alteração de branding e cores via variáveis CSS.

3.6 Modelagem do Sistema

A modelagem permitiu a visualização das interações e da estrutura de dados necessária para a solução.

3.6.1 Diagrama de Casos de Uso

O diagrama de casos de uso (Figura 1) ilustra as interações entre o Administrador, o Leitor e as automações de sistema que disparam o processo de ISR (6, 5).

3.6.2 Modelagem de Dados (ER)

A modelagem de dados (Figura 2) detalha a estrutura do banco de dados SQLite, com foco na auto-relação para suportar a hierarquia de conteúdos através do Prisma ORM (4).

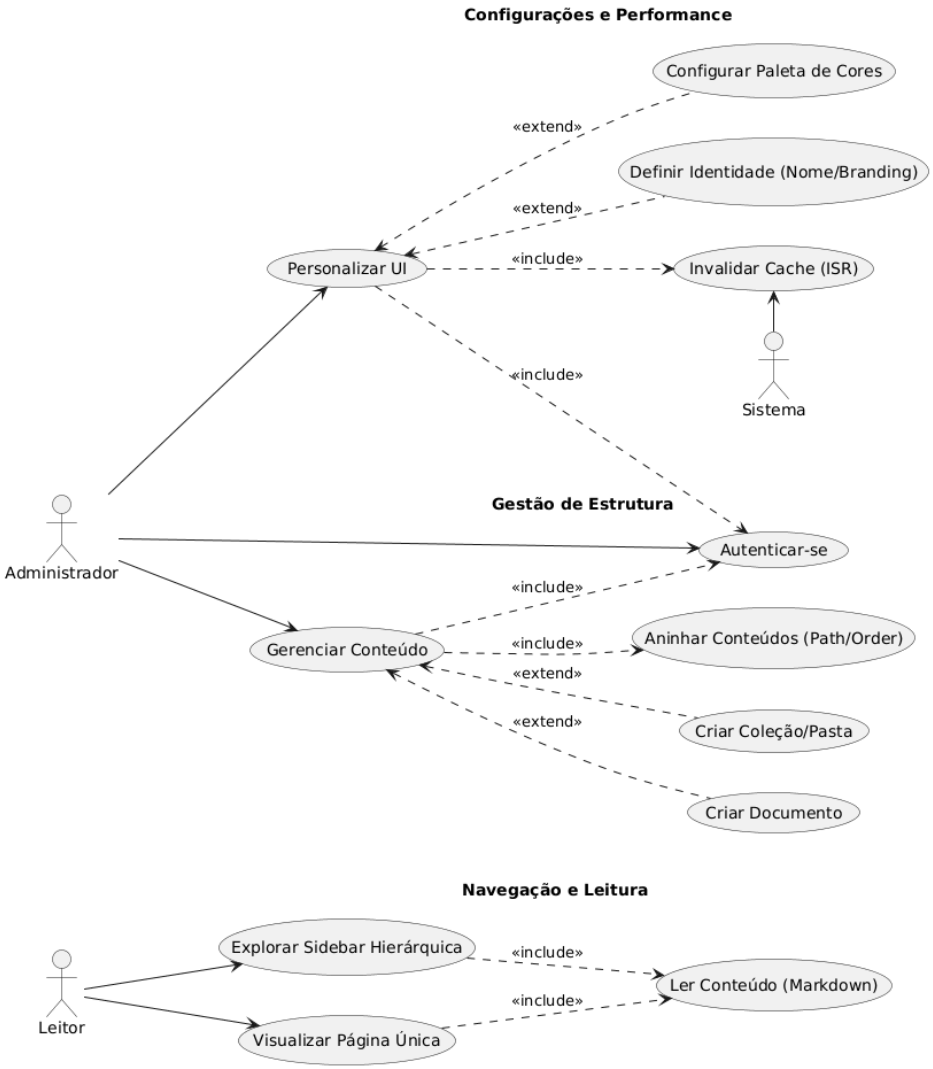


Figura 1 – Diagrama de Casos de Uso do sistema Vitrine.

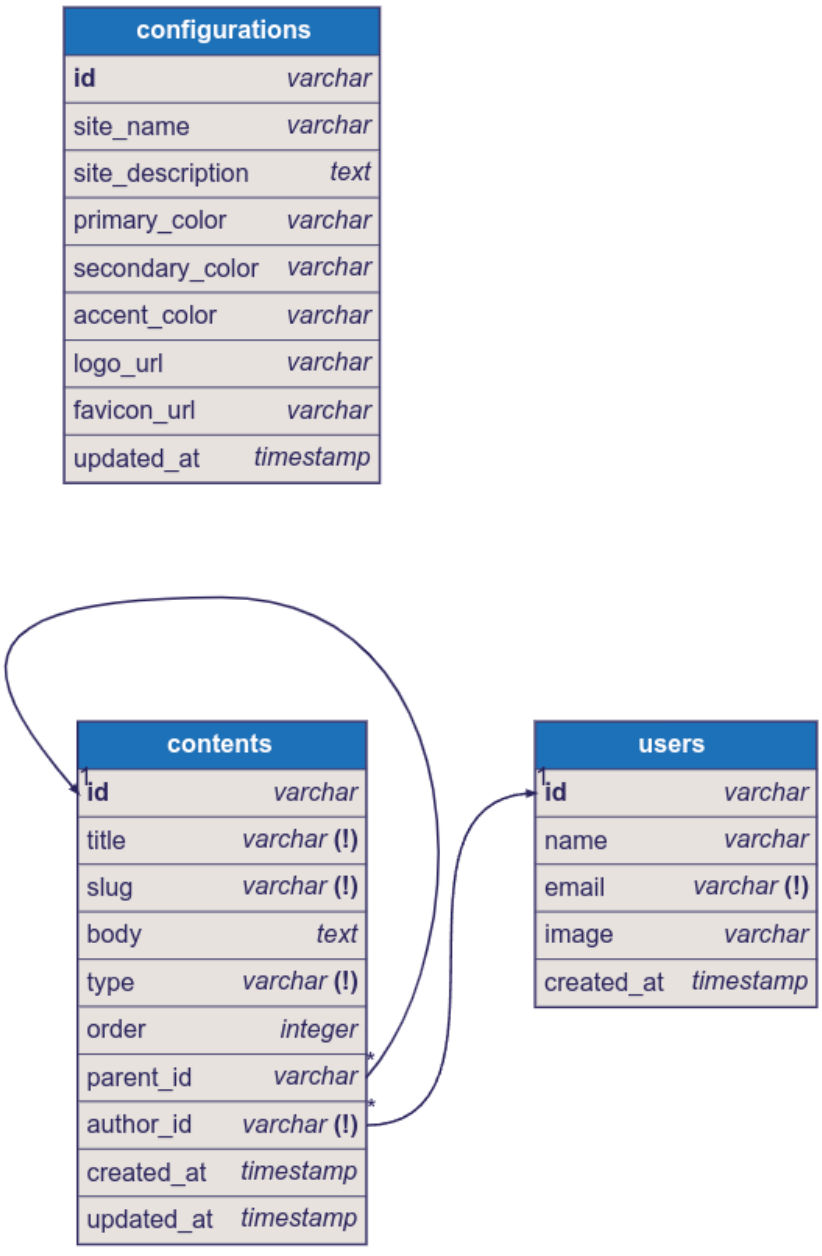


Figura 2 – Diagrama de Entidade-Relacionamento do sistema Vitrine.

¸hapterDesenvolvimento do Sistema

Neste capítulo, detalham-se as etapas de construção do sistema Vitrine, abrangendo desde a concepção visual por meio de protótipos até a implementação técnica da arquitetura baseada em Next.js e SQLite.

3.7 Arquitetura Técnica

A arquitetura do Vitrine foi planejada sob o paradigma Jamstack, visando desacoplar a gestão de dados da camada de apresentação (5).

- **Front-end**: Desenvolvido com o framework **Next.js**, utilizando renderização híbrida para otimizar o SEO e a experiência do usuário (5).
- **Persistência**: Utilização do **Prisma ORM** em conjunto com o **SQLite**. A escolha do SQLite como banco de dados baseado em arquivo é estratégica para garantir a portabilidade e reduzir a complexidade de manutenção para o usuário final (4).
- **Autenticação**: Implementada via **Auth.js**, garantindo um sistema de login simplificado e seguro para o gerenciamento de conteúdos.

3.8 Interfaces e Prototipagem

A etapa de prototipagem de baixa fidelidade permitiu validar a jornada do usuário e a disposição dos elementos antes da codificação final (6). Para ilustrar o funcionamento do sistema e a versatilidade da ferramenta, são apresentadas a seguir as quatro interfaces principais que compõem a solução.

3.8.1 Visualização do Sistema com Conteúdo (Instância FromCaio)

A Figura 3 demonstra uma instância do sistema em operação, preenchida com conteúdos de programação. O exemplo exibe cursos organizados em diferentes linguagens, demonstrando como a plataforma atua como uma vitrine de conhecimento técnico unificada.

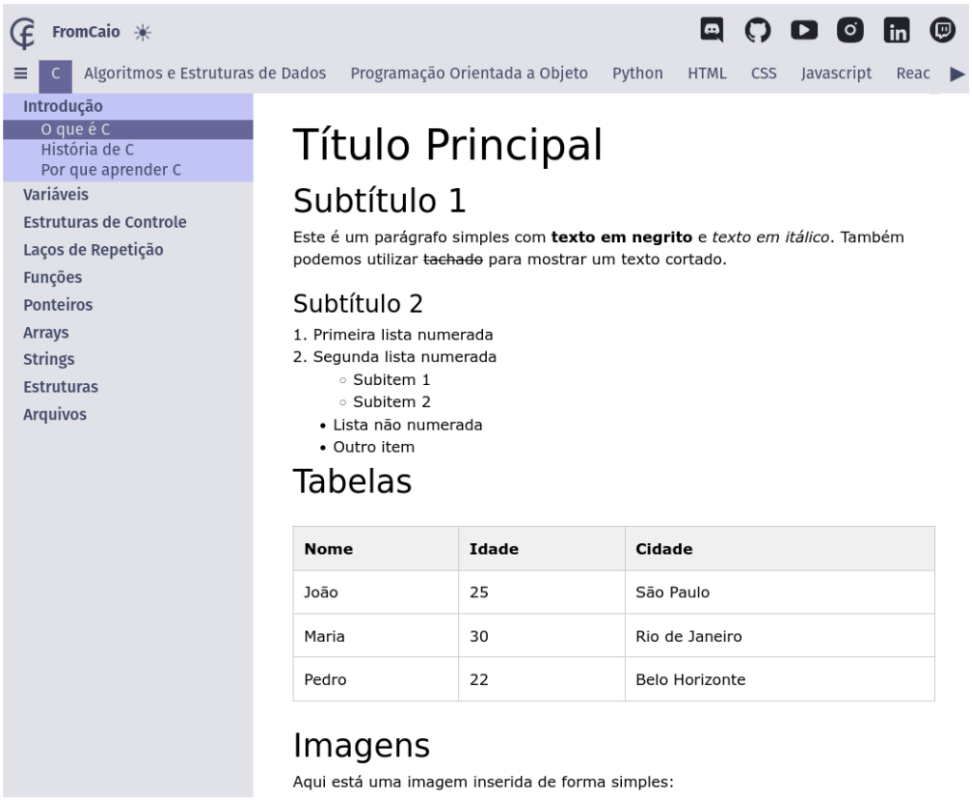


Figura 3 – Instância do sistema Vitrine populada com cursos de programação.

3.8.2 Gestão e Organização de Conteúdos

O sistema permite uma estrutura hierárquica onde coleções e arquivos são aninhados. A Figura 4 apresenta a interface administrativa de organização, onde o usuário gerencia a árvore de conteúdos e define a precedência de exibição de cada módulo (6).

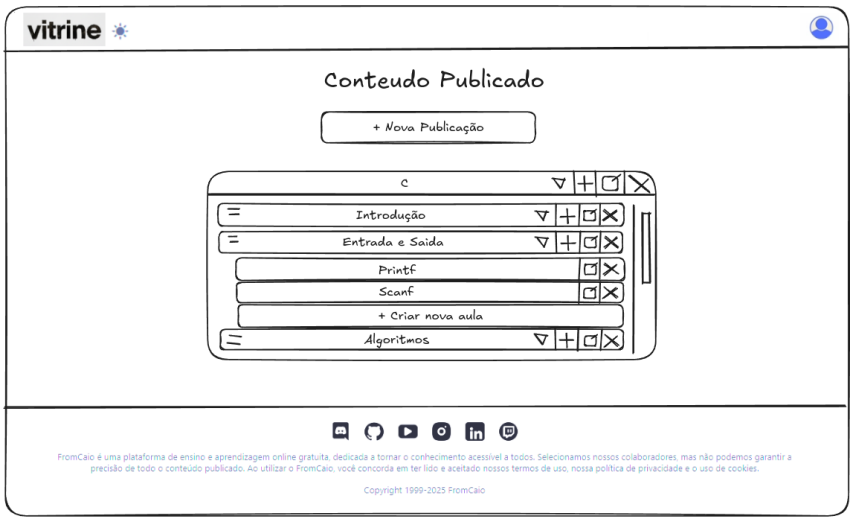


Figura 4 – Interface de organização de documentos e coleções aninhadas.

3.8.3 Customização Dinâmica

A tela de customização (Figura 5) permite ao administrador alterar a identidade visual, incluindo o logotipo e a paleta de cores primária. Essas alterações manipulam variáveis CSS em tempo de execução (5).



Figura 5 – Interface de personalização de cores e identidade visual.

3.8.4 Editor de Escrita Científica

A interface do editor (Figura 6) foi priorizada para suportar a escrita acadêmica (1). Ela permite o uso de Markdown integrado a fórmulas *LaTeX* e blocos de código com realce de sintaxe, garantindo que a formatação técnica seja preservada.

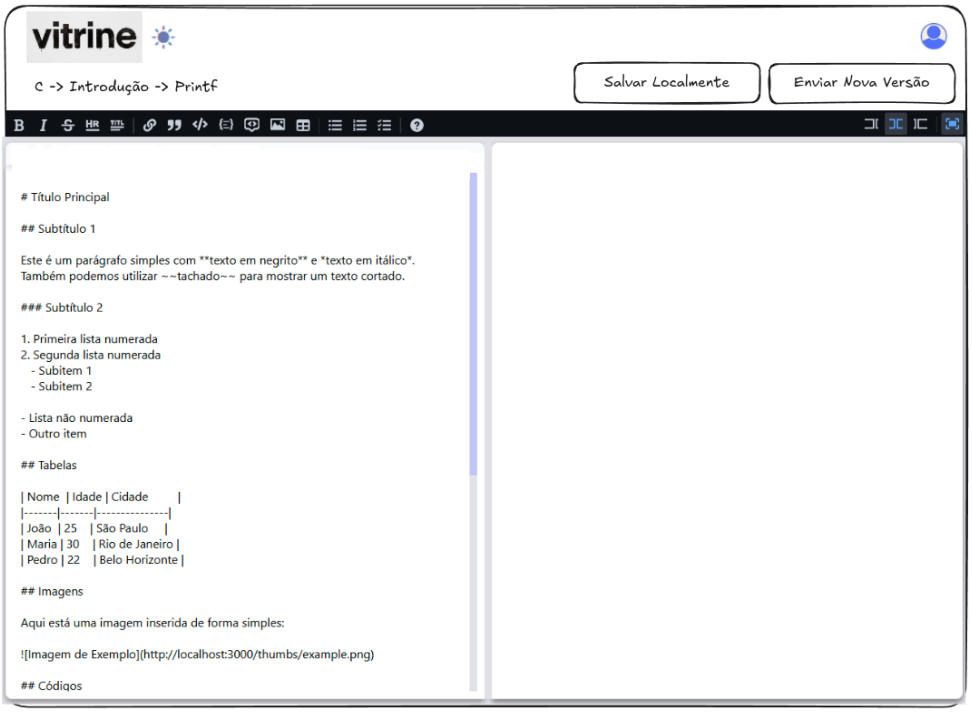


Figura 6 – Interface do editor de texto com suporte a elementos técnicos.

3.9 Implementação das Funcionalidades Centrais

A construção seguiu um modelo incremental, garantindo que cada funcionalidade fosse validada de ponta a ponta (6).

3.9.1 Sistema de Cores e ISR

Uma das implementações centrais é a manipulação de variáveis CSS. As preferências de cores são persistidas no SQLite e refletidas instantaneamente através de *Incremental Static Regeneration* (ISR), unindo a performance de sites estáticos com a flexibilidade de sistemas dinâmicos (5).

4 Conclusão

Este relatório detalhou o ciclo de desenvolvimento do sistema **Vitrine**, uma plataforma CMS *open source* voltada para a democratização da publicação acadêmica independente. Ao longo do período de iniciação científica, entre setembro de 2025 e fevereiro de 2026, foi possível converter o desafio da dependência de plataformas proprietárias em uma solução funcional que prioriza a soberania do autor sobre sua produção intelectual.

4.1 Resultados Alcançados

Os objetivos propostos foram integralmente atingidos através de uma abordagem de desenvolvimento incremental (6). O sistema consolidou o equilíbrio entre modernidade tecnológica e portabilidade ao utilizar o framework **Next.js** e o banco de dados **SQLite** (5, 4). Entre as principais conquistas técnicas, destacam-se:

- A implementação de um editor *markdown* capaz de renderizar com precisão elementos essenciais da escrita científica, como fórmulas em *LaTeX* e blocos de código com realce de sintaxe;
- O desenvolvimento de uma interface de personalização visual dinâmica, permitindo que o usuário gerencie a estética do site via variáveis CSS persistidas em banco de dados, com reflexão instantânea através de *Incremental Static Regeneration* (ISR) (5);
- A garantia da perenidade do conteúdo digital, uma vez que a arquitetura baseada em arquivos portáteis assegura que o pesquisador detenha a posse integral de seus dados (4).

4.2 Limitações e Desafios

Apesar dos avanços, o projeto encontrou fronteiras delimitadas pelo prazo de execução. A modelagem da interface restringiu-se a protótipos de baixa fidelidade, focando na funcionalidade e usabilidade em detrimento de um refinamento visual mais exaustivo (6). Além disso, o sistema de autenticação via *Auth.js*, embora seguro e funcional para o gerenciamento básico, ainda pode ser expandido para suportar múltiplos níveis de permissão em cenários colaborativos.

4.3 Considerações Finais e Trabalhos Futuros

A experiência de desenvolver o Vitrine reafirmou a premissa de que sistemas de informação devem atuar como facilitadores da disseminação de conhecimento, e não como obstáculos técnicos (3). O projeto entrega uma base sólida e moderna que elimina o trabalho repetitivo de configuração estrutural, permitindo que o foco permaneça na produção de conteúdo qualificado.

Como propostas para a continuidade e expansão do projeto, pretende-se:

- Produzir documentação técnica detalhada e tutoriais em vídeo demonstrando o processo de clonagem do repositório e *deployment* na plataforma Vercel;
- Lançar o site oficial do projeto, denominado **Vitrine**, que servirá como repositório central para os tutoriais, documentação e instâncias de demonstração;
- Explorar a integração com repositórios de dados abertos para a citação automática de publicações;
- Expandir a biblioteca de componentes visuais, permitindo maior customização da interface;
- Implementar a renderização nativa de gráficos para funções de uma variável (gráficos 2D).

O Vitrine encerra este ciclo de iniciação científica não apenas como uma ferramenta técnica, mas como um manifesto em prol da autonomia digital e da democratização do saber científico na rede.

Referências

1 GINIGE, A.; MURUGESAN, S. Web engineering: An introduction. *IEEE multimedia*, IEEE, v. 8, n. 1, p. 14–18, 2001.

2 MIKKONEN, T.; TAIVALSAARI, A. The mashware challenge: bridging the gap between web development and software engineering. In: *Proceedings of the FSE/SDP workshop on Future of software engineering research*. [S.l.: s.n.], 2010. p. 245–250.

3 RAINER, K.; PRINCE, B. *Introduction to Information Systems*. 8th. ed. New York: Wiley, 2019.

4 INC., P. D. *Prisma Documentation*. 2026. Disponível em: <https://www.prisma.io/docs>. Acesso em: 2026-03-27.

5 VERCEL. *Next.js Documentation*. 2026. Disponível em: <https://nextjs.org/docs>. Acesso em: 2026-03-27.

6 SOMMERVILLE, I. *Engenharia de Software*. 9. ed. São Paulo: Pearson, 2011.